Task 1 : Access and Print the Element at a Given Index in an Array

Aim

To access and print the element at a specified index in an array using Java.

Algorithm

Start.

Read the size of the array.

Read the array elements.

Read the index to access.

Check whether the index is valid.

If valid, print the element at that index.

Otherwise, display "Invalid Index".

Stop.

Program (Java)

```
import java.util.Scanner;

public class ArrayIndex {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter size of array: ");
        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter array elements:");
```

```
        for(int i=0;i<n;i++)
            arr[i]=sc.nextInt();

        System.out.print("Enter index: ");
        int index=sc.nextInt();

        if(index>=0 && index<n)
            System.out.println("Element = "+arr[index]);
        else
            System.out.println("Invalid Index");
    }
}
```

Sample Output

```
Enter size of array: 5
Enter array elements:
10 20 30 40 50
Enter index: 2
Element = 30
```

Result

The program successfully accesses and prints the element at the given array index.

Task 2 : Search for an Element Using Binary Search

Aim

To search for a given element in a sorted array using the Binary Search algorithm.

Algorithm

Read the size of the sorted array.

Read the array elements.

Read the key to search.

Set low = 0 and high = n-1.

Repeat while low <= high:

Find mid = (low + high) / 2.

If arr[mid] == key, print the position.

If key < arr[mid], search the left half.

Otherwise, search the right half.

If the element is not found, display an appropriate message.

Stop.

Program (Java)

```
import java.util.Scanner;

public class BinarySearchDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter size: ");
        int n = sc.nextInt();

        int[] arr = new int[n];
```

```
    System.out.println("Enter sorted array:");

    for(int i=0;i<n;i++)
        arr[i]=sc.nextInt();

    System.out.print("Enter element to search: ");
    int key=sc.nextInt();

    int low=0, high=n-1;
    boolean found=false;

    while(low<=high){

        int mid=(low+high)/2;

        if(arr[mid]==key){
            System.out.println("Element found at index "+mid);
            found=true;
            break;
        }
        else if(key<arr[mid])
            high=mid-1;
        else
            low=mid+1;
    }

    if(!found)
        System.out.println("Element not found");
}
```

}

Sample Output

Enter size: 6

Enter sorted array:

10 20 30 40 50 60

Enter element to search: 40

Element found at index 3

Result

The program successfully searches the given element using the Binary Search algorithm.

Task 3 : Find the Maximum Element in an Array

Aim

To find the maximum element in an array of integers.

Algorithm

Read the array size.

Read the array elements.

Assume the first element is the maximum.

Compare each remaining element with the current maximum.

Update the maximum if a larger element is found.

Print the maximum element.

Stop.

Program (Java)

```
import java.util.Scanner;

public class MaximumElement {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter size: ");
        int n=sc.nextInt();

        int[] arr=new int[n];

        System.out.println("Enter elements:");

        for(int i=0;i<n;i++)
            arr[i]=sc.nextInt();

        int max=arr[0];
```

```
        for(int i=1;i<n;i++){

            if(arr[i]>max)
                max=arr[i];
        }

        System.out.println("Maximum Element = "+max);
    }
}
```

Sample Output

Enter size: 5

Enter elements:

10 35 12 90 45

Maximum Element = 90

Result

The program successfully finds and displays the maximum element in the array.

Task 4 : Given an Array of Integers and a Positive Integer K (Rotate Array)

Note: Since the problem statement is incomplete ("Given an array of integers and a positive integer K"), a common lab task is rotating the array to the right by K positions.

Aim

To rotate the elements of an array to the right by K positions.

Algorithm

Read the size of the array.

Read the array elements.

Read the value of K.

Compute K = K % n.

Print the last K elements first.

Print the remaining elements.

Stop.

Program (Java)

```
import java.util.Scanner;

public class RotateArray {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter size: ");
        int n=sc.nextInt();

        int[] arr=new int[n];

        System.out.println("Enter elements:");

        for(int i=0;i<n;i++)
            arr[i]=sc.nextInt();
```

```
        System.out.print("Enter K: ");
        int k=sc.nextInt();

        k=k%n;

        System.out.println("Rotated Array:");

        for(int i=n-k;i<n;i++)
            System.out.print(arr[i]+" ");

        for(int i=0;i<n-k;i++)
            System.out.print(arr[i]+" ");
    }
}
```

Sample Output

Enter size: 5

Enter elements:

1 2 3 4 5

Enter K: 2

Rotated Array:

4 5 1 2 3

Result

The program successfully rotates the array to the right by K positions and displays the rotated array.

Task 5 : Print All Possible Pairs of Elements from an Array

Aim

To write a Java program to print all possible pairs of elements from a given array of size n.

Algorithm

Start.

Read the size of the array n.

Read the array elements.

Use two nested loops:

Outer loop: i = 0 to n - 2

Inner loop: j = i + 1 to n - 1

Print the pair (arr[i], arr[j]).

Stop.

Program (Java)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println("Possible Pairs:");
        for (int i = 0; i < n - 1; i++) {
            for (int j = i + 1; j < n; j++) {
                System.out.println("(" + arr[i] + ", " + arr[j] + ")");
            }
        }
    }
}
```

Sample Input

```
4
1 2 3 4
```

Sample Output

```
Possible Pairs:
```

(1, 2)

(1, 3)

(1, 4)

(2, 3)

(2, 4)

(3, 4)

Result

Thus, the Java program to print all possible pairs of elements from an array was implemented and executed successfully.